# Analysis of Machine Learning Models for App Type and Rating Classification in the Google Play Store

**Azmayeen Aftab**
*Dept of Computer Science and Engineering*
*BRAC University*
Dhaka, Bangladesh
azmayeen.aftab@g.bracu.ac.bd

**Labiba Binte Arif**
*Dept of Computer Science and Engineering*
*BRAC University*
Dhaka, Bangladesh
labiba.binte.arif@g.bracu.ac.bd

**Naim Mahmud Peyas**
*Dept of Computer Science and Engineering*
*BRAC University*
Dhaka, Bangladesh
naim.mahmud.peyas@g.bracu.ac.bd

***Abstract*—The Google Play Store hosts millions of applications spanning a wide range of categories, and for any developer looking to launch a new app, understanding what factors drive success is critical. This study implements and compares seven machine learning models - Logistic Regression, Linear Regression, Decision Tree, Random Forest, XGBoost, KNN, and a Voting Classifier/Regressor - to predict two key outcomes: whether an app is free or paid and its price if paid, and whether an app is recommended based on its user rating. Post-launch features such as Reviews, Installs, and Rating were excluded from the feature set to prevent data leakage and ensure predictions reflect only information available at the time of app release. The results show that nearly all models achieved near-perfect classification performance on the Free/Paid task, with most reaching perfect accuracy of 1.000 and KNN being the only exception at 0.985. For the recommendation task, Logistic Regression led with 80.97% accuracy while Random Forest achieved the strongest F1 score of 75.44%, reflecting better balance in handling class imbalance. A key finding is that app metadata alone carries limited predictive power for rating-based classification, suggesting that richer data sources such as user review text would be needed to meaningfully improve recommendation prediction.**

***Index Terms*—Google Play Store, Application, User Features, Machine Learning, Logistic, Random Forest, Accuracy, XG Boost, Version, Ratings**

## I. Introduction

In the Android Market, Google Play Store developers offer a wide range of applications - both free and paid - to users. Free apps are particularly attractive as they allow users to try them instantly without any upfront cost, while paid apps require users to carefully consider their investment before downloading [1]. Free apps often follow either the freemium model or the ads-business model where the revenue is generated through in-app charges and ads. Moreover, even if an app is free, it has hidden ad charges and as study suggests, by forming Google family groups, it is possible to share the purchased apps. This creates an issue for both the app developer and user to balance the actual charges. Accurately predicting market trends is critical to the success of applications which can be done by evaluating various factors such as application name, pricing, reviews, and category. Prediction of which types of apps are most likely to work better in the market is possible [2]. A longitudinal study of the Google app store dataset analysis suggests that if the features are tracked down correctly, some common factors can be found. For instance, spikes in the number of positive or negative reviews and the number of downloads received by an app can indicate app search optimization campaigns launched by fraudsters recruited through crowd sourcing sites [3]. The features and correlations that are extracted from the analysis can be used to apply on supervised models. It can improve user experience and also detect the success rate of an app. User satisfaction can also be detected by sentiment analysis or user provided reviews. By incorporating user feedback with machine learning models, it's possible to offer insights on user recommendation or the app version and its predicted price [4].

This research aims to find out the success of google apps based on the common features found through analysis. This draws the attention to user sentiments that can help us to detect the ratings and how the ratings are correlated with the app being free or paid. By implementing machine learning models, it is possible to separate the target columns (price and rating) and thus draw correlation of the features affecting it. This report can provide accuracy of different models thus conclude which model can work better for a developer to detect success rate and understand user perspective.

## II. Literature Review

Considerable research has been conducted on Google Play Store analysis, with most studies emphasizing the role of user reviews and ratings in shaping app success. Khan et al. (2026) directed attention toward low-rated

apps on the Amazon App store, arguing that this underexplored segment reveals valuable information about software quality and user experience issues. Their dataset of 79,821 reviews from 64 low-rated apps, with 6,000 manually annotated across six issue categories, provides a foundation for machine learning models that classify user feedback and identify the root causes of poor ratings [5]. Tegegnie (2026) examined mobile app reviews in an Ethiopian context using a multi-task learning framework on 10,200 Google Play Store reviews, simultaneously addressing sentiment classification, feedback categorization, and rating prediction. Convolutional neural networks achieved the strongest results across all three tasks [6] , demonstrating that deep learning can effectively capture linguistic patterns even in less-resourced language settings and offering practical insights for developers seeking to leverage community feedback [7].

Among the features available in app store datasets, app type, installs, user ratings and reviews show the strongest correlations with app success. Maringka et al. (2021) compared Random Forest against Decision Trees and Logistic Regression for rating classification, finding that Random Forest yielded the highest accuracy among the models tested [8]. Ramakrishnan et al. (2023) reached a similar conclusion, noting that ensemble methods tend to outperform individual classifiers due to their ability to capture complex feature relationships, and further emphasized that user ratings and reviews are the primary indicators of app success [9].

The relationship between quantitative features and app ratings was explored by Seo et al. (2020) through regression models, neural networks, and decision trees, revealing that review counts and install numbers carry the strongest positive correlations with ratings [10]. Notably, text-based features such as review sentiment were found to contribute less than numerical metrics, suggesting that quantitative data holds more predictive weight in regression-based analysis. Taking the opposite stance, Deepika et al. (2024) centered their work on text-based features using Gradient Boosting and Artificial Neural Networks, finding that ensemble and gradient boosting methods outperform traditional classifiers while still confirming that user reviews remain central to predicting app success [11]. Sai Kiran et al. (2022) further supported these findings, demonstrating that Random Forest consistently outperforms individual classifiers when evaluating apps across price, rating, and review metrics [12].

Exploratory data analysis on 5,223 apps and over 170,000 reviews was carried out by Mueez et al. (2018), applying Random Forest, XGBoost, KNN, and SVM for success classification. XGBoost and ensemble methods achieved the highest accuracy at approximately 86%, and the analysis revealed that free apps dominate in both count and installs, while install counts showed only weak correlation with most numeric features [13].

Building on a similar premise, Sarker et al. (2023) developed a recommendation system using TF-IDF vectorization and sentiment analysis on Amazon mobile phone reviews, where Random Forest achieved the best accuracy among traditional models at 76% [14]. Deep learning models further pushed this to 80%, reinforcing that more complex architectures better capture the nuances of review-based rating prediction.

## III. Methodology

### A. Dataset Description

The dataset used in this project is the Google Play Store Apps dataset obtained from Kaggle, consisting of approximately 10,000 application records across 13 features. Each record represents a unique application and includes attributes such as category, rating, number of reviews, app size, number of installs, type (free or paid), price, content rating, genres, last updated date, current version, and minimum Android version required. The dataset was selected because it contains both numerical and categorical data and directly supports the two prediction targets of this study - app type and pricing for Task 1, and rating-based recommendation for Task 2. It also provides sufficient volume and feature diversity to meaningfully compare multiple machine learning models across both classification and regression objectives. It is important to note that features such as Reviews, Installs, and Rating reflect post-launch metrics that are updated over time alongside user engagement. Including these as predictive features would constitute data leakage, as they would not be available at the moment an app is released. Accordingly, these features were excluded from the model feature sets, and predictions were built solely on features available prior to or at the time of launch, more specifically Size, Price, Category, Content Rating, and Genres.

### B. Data Preprocessing

The raw dataset required several cleaning and transformation steps before it could be used for model training:

- **Irrelevant columns removed** - App name, Last Updated, Current Version, and Android Version were dropped as they do not contribute meaningful predictive information. Rows with Type values other than Free or Paid were also filtered out.
- **Duplicate removal** - Duplicate entries were dropped to prevent any single app from disproportionately influencing the models.
- **Price column** - Dollar signs were stripped and values were converted to float. Missing values were filled as zero.
- **Installs column** - Plus signs and commas were removed before converting to numeric type.
- **Size column** - Values were expressed in either megabytes (M) or kilobytes (k). A custom function

was applied to standardize all values to MB, with entries such as Varies with device set to NaN.

- **Missing values** - All numeric columns with missing values were filled with the column median. Missing categorical values were filled with the mode.
- **One-hot encoding** - The Category, Content Rating, and Genres columns were encoded using get_dummies, converting them into binary columns suitable for model input. This expanded the feature space significantly but is necessary as machine learning models cannot process raw string categories.
- **Feature selection** - Reviews, Installs, and Rating were excluded from the model feature sets as they are post-launch metrics not available at the time of app release. Final features used for modeling were Size, Price, Category, Content Rating, and Genres.

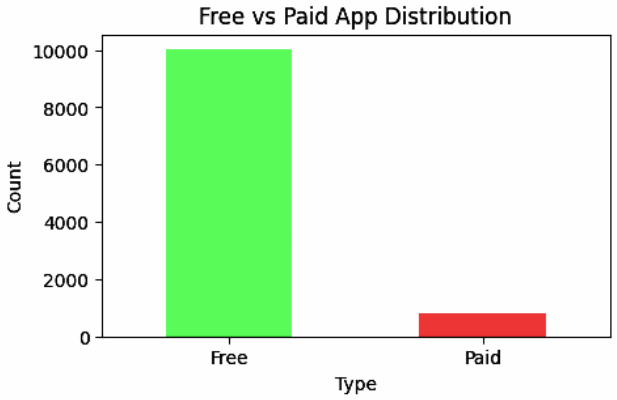



Fig. 1. Free vs Paid App Distribution

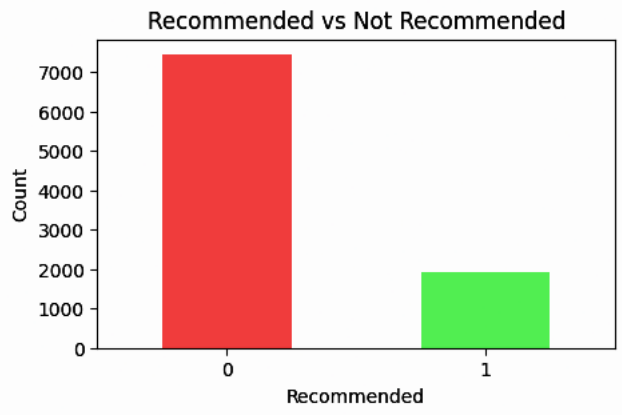



Fig. 2. Recommended vs Not Recommended

### C. Models implemented by Learning Phase

1) Logistic Regression
Logistic regression is a supervised machine learning algorithm used for classification, predicting the probability of categorical outcomes. As one of our targets is to detect app version i.e free or paid, we denoted Free as 0 and Paid as 1 rather than using continuous numbers. Similarly, for rating detection , we termed above 4.5 as Recommended and below that as not recommended. It maps inputs to a sigmoid function limiting output values between 0 and 1, facilitating binary classification.

2) Linear Regression
Linear Regression is the simplest regression model, establishing a linear relationship between input features and a continuous target variable by fitting a straight line that minimizes the sum of squared errors. Unlike the classification models used in Task 1a and Task 2, Linear Regression outputs a continuous numerical value, making it suitable exclusively for Task 1b where the goal is to predict the actual price of a paid application. It serves as the baseline regression model against which the performance of more complex regressors such as Random Forest,

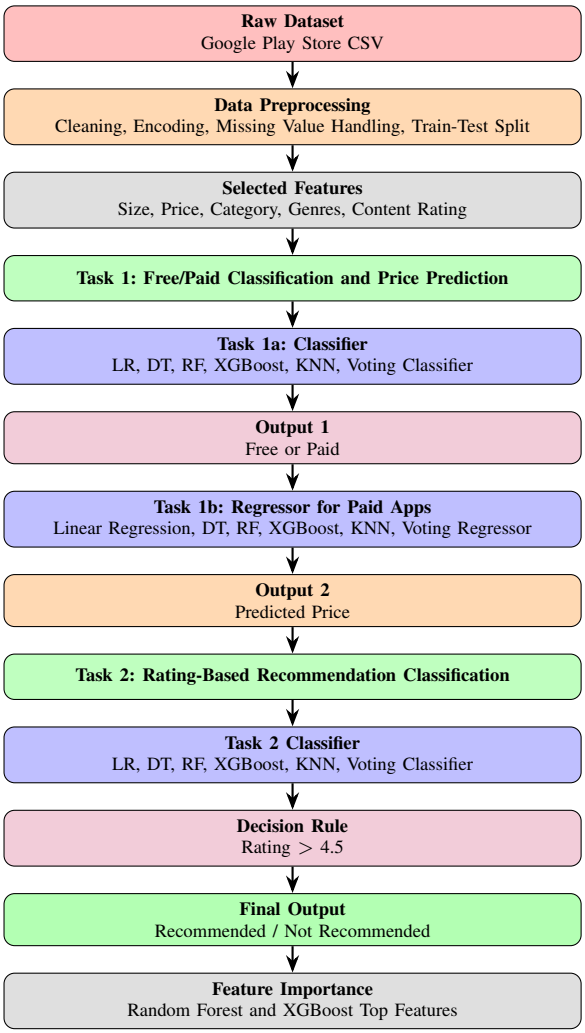



Fig. 3. Machine learning pipeline for Google Play Store app classification, price prediction, and recommendation.

XGBoost, and KNN are compared.

3) Random Forest
   Random Forest is an ensemble machine learning model that creates multiple decision trees, which in our case is 100, that works for both classification and regression tasks. Here, we measure the errors using MAE (Mean Absolute Error) and RMSE (Root Mean Squared Error). Based on these error values, it is easier to make predictions on how the model is performing.
4) XGBoost
   XGBoost is an optimized distributed gradient boosting library designed to be highly efficient, flexible, and portable. It implements machine learning algorithms under the Gradient Boosting framework. XGBoost provides a parallel tree that powers many data science problems in a fast and accurate way.
5) Decision Tree
   A Decision Tree in Machine Learning is used in both classification and regression trees for prediction. Classification Trees are used for predicting categorical outcomes like spam or not spam whereas regression trees act for continuous values. Instead of assigning categories, it provides numerical predictions based on the input features.
6) KNN
   K-Nearest Neighbors (KNN) is a supervised machine learning algorithm used for classification and regression which predicts a new data point's label based on the majority class of its closest neighbors. It forms cluster by similar points found near one another making assumptions. For classification problems, a class label is assigned on the basis of a majority vote.
7) Voting Classifier
   Voting Classifier / Voting Regressor is an ensemble technique that combines the predictions of all five individual models. For classification tasks, both hard voting where the final prediction is the majority class, and soft voting where predicted probabilities are averaged - were implemented and compared. For the regression task, a Voting Regressor was used which averages the price predictions of all five regressors. This approach aims to produce more stable and robust predictions by offsetting the weaknesses of individual models.

## Result and Evaluation

### A. Feature Importance

The correlation matrix below shows the relationships between the key numerical features in the dataset. Price and Size show very weak correlations with all other features, indicating they carry largely independent information. The remaining features such as Category, Content Rating, and Genres are categorical and were one-hot encoded, making them the primary drivers of model predictions as confirmed by the Random Forest and XGBoost feature importance plots, where category-based features consistently ranked among the top contributors across all three tasks.

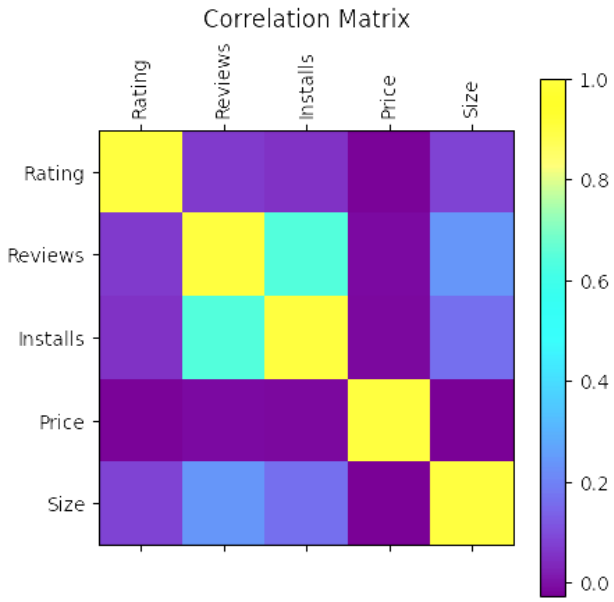



Fig. 4. Correlation Matrix

### B. Performance Metrics

For the first target, we detect if an app is free or paid and then if it's paid, we detect the price of the model. To detect the version as shown in Table I, models including Logistic Regression, Decision Tree, Random Forest, XGBoost, and the Voting Classifier achieved perfect accuracy of 1.000, while KNN achieved slightly lower accuracy of 0.985. Additionally, evaluation metrics such as Precision, Recall, and F1-score were considered. All models achieved consistently high values across these metrics, indicating strong and balanced classification performance for both Free and Paid classes.

TABLE I
Task 1a – Free/Paid Classification Results

| Model | Accuracy | Precision | Recall | F1 Score |
|---|---|---|---|---|
| Logistic Regression | 1.000 | 1.0000 | 1.000 | 1.0000 |
| Decision Tree | 1.000 | 1.0000 | 1.000 | 1.0000 |
| Random Forest | 1.000 | 1.0000 | 1.000 | 1.0000 |
| XGBoost | 1.000 | 1.0000 | 1.000 | 1.0000 |
| KNN | 0.985 | 0.9852 | 0.985 | 0.9841 |
| Voting Classifier | 1.000 | 1.0000 | 1.000 | 1.0000 |

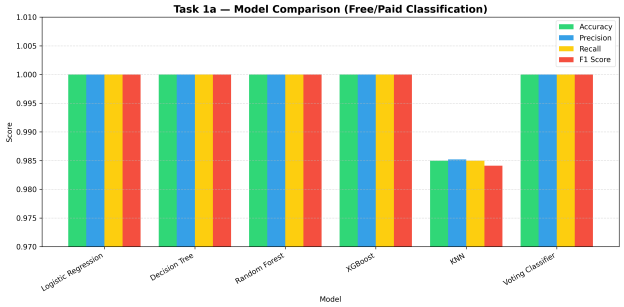



Fig. 5. Task 1a — Model Comparison (Free/Paid Classification)

As for the paid apps predictions, we use regression models that would detect the price for us which is a numerical value. We measure the model accuracies here, calculating the errors like MAE or RMSE and $R^2$ as evaluation metrics. XGBoost has the best performance here giving low error values and nearly perfect $R^2$ values. However, this nearly perfect values suggest the task may be over simplified.

TABLE II
TASK 1B – PRICE PREDICTION RESULTS

| Model | MAE | RMSE | R2 |
|---|---|---|---|
| Decision Tree | 0.0620 | 0.4329 | 0.9999 |
| Linear Regression | 0.0000 | 0.0000 | 1.0000 |
| Random Forest | 0.0303 | 0.1679 | 1.0000 |
| XGBoost | 0.0459 | 0.4111 | 0.9999 |
| KNN | 0.5860 | 1.1926 | 0.9995 |
| Voting Regressor | 0.1359 | 0.3427 | 1.0000 |

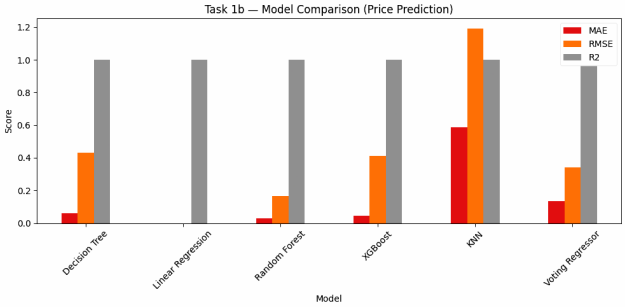



Fig. 6. Task 1b — Model Comparison (Price Prediction)

For the second target, we aim to detect if an app is recommended or not based on our fixed threshold. For that, we find connections with the numbers of ratings. When models were implemented based on the target variable rating, it predicts the Accuracy, Precision, Recall, and F1-score, as shown in Table III. Logistic Regression achieved the highest accuracy (0.8097), closely followed by XGBoost and the Voting Classifier. However, as F1-scores are relatively lower, hence due to the class imbalance, it's risky to rely on accuracy alone. Thus, predicting recommendation values based on the given dataset is a bit tricky compared to our previous task.

TABLE III
TASK 2 – RECOMMENDED CLASSIFICATION RESULTS

| Model | Accuracy | Precision | Recall | F1 Score |
|---|---|---|---|---|
| Logistic Regression | 0.8097 | 0.6556 | 0.8097 | 0.7246 |
| Decision Tree | 0.8029 | 0.7130 | 0.8029 | 0.7296 |
| Random Forest | 0.7762 | 0.7414 | 0.7762 | 0.7544 |
| XGBoost | 0.8083 | 0.7256 | 0.8083 | 0.7274 |
| KNN | 0.7971 | 0.7426 | 0.7971 | 0.7530 |
| Voting Classifier | 0.8068 | 0.7263 | 0.8068 | 0.7301 |

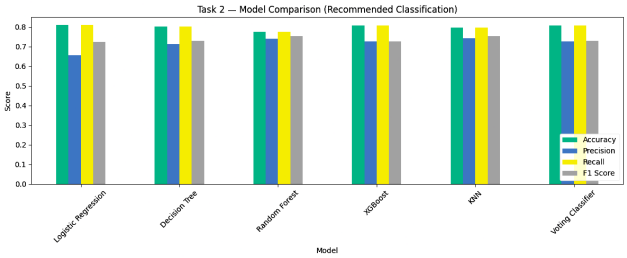



Fig. 7. Task 2 — Model Comparison (Recommended Classification)

## DISCUSSION

Seven machine learning models were implemented and evaluated across two classification tasks and one regression task. The results reveal clear patterns in how different model types perform depending on the nature of the prediction problem.

For Task 1a, most models including Logistic Regression, Decision Tree, Random Forest, XGBoost, and the Voting Classifier achieved perfect accuracy of 1.000, with KNN slightly behind at 0.985. However, this near-perfect performance raises an important concern since app Type is directly derived from the Price column, where free apps always have a price of 0 and paid apps always have a price greater than 0, the model is effectively learning a deterministic rule rather than a generalizable pattern. This means the high accuracy reflects direct feature-to-target mapping rather than true learned behavior.

For Task 1b, all regression models performed exceptionally well, with XGBoost achieving an MAE of 0.0459 and $R^2$ of 0.9999, and Linear Regression reaching a perfect $R^2$ of 1.0000 with near-zero MAE. While these results appear strong, they likely reflect an overly simplified price distribution in the dataset rather than genuine predictive power. KNN was the weakest regressor with the highest error values of MAE 0.5860 and RMSE 1.1926.

For Task 2, results were more modest due to severe class imbalance - very few apps exceed the 4.5 rating threshold. Logistic Regression led with 80.97% accuracy while Random Forest achieved the strongest F1 score of 75.44%, reflecting better balance in handling the minority class. The gap between accuracy and F1 scores across all models confirms that accuracy alone is misleading here, and that models tend to favor the majority class.

The Voting Classifier, tested under both hard and soft voting, delivered consistent but average results across all

tasks - confirming that ensemble combinations improve stability without necessarily surpassing the best individual model. KNN was the weakest performer overall, particularly in regression, consistent with the expected limitations of distance-based models in high-dimensional feature spaces.

## CONCLUSION AND FUTURE WORKS

This study compared seven machine learning models across two classification tasks and one regression task on the Google Play Store dataset. All models achieved very low error and high $R^2$ value reliably on the first part of target 1 (Free/Paid classification) and price detection, with KNN achieving a slightly lower accuracy and F1 score of 98.5% and 0.9841 respectively. In case of price detection, we measured the errors where Linear Regression shows near zero error and a perfect value of $R^2 = 1.00$, assuming the dataset might be over-simplified. Target 2 (recommendation prediction), although having class imbalance, resulted in the highest accuracy with Logistic Regression leading at 80.97% accuracy. This was the most challenging across all models, as for the recommended class we had very few data resulting in class imbalance. We also had to drop certain features which were causing data leakage such as reviews, installs and rating before the post launch. The analysis also confirms that free apps dominate the market, and that install counts and review numbers are more strongly linked to app success than other metadata features.

Limitations-

A key limitation is that app metadata alone carries weak predictive power for rating-based classification, suggesting that additional data sources would be needed to meaningfully improve performance on target 2.

In case of Task(1a), that is free or paid classification, we were already using a conditional where if price $> 0$, then the app is paid, else it's not. Hence, the model is not learning anything from it as it is a given condition. Although, this creates low noise and a perfect accuracy due to a given rule, ultimately this is direct mapping instead of trained model behavior.

For future work, incorporating review text and sentiment analysis as features could strengthen prediction accuracy, particularly for the recommendation task. Applying class balancing techniques such as SMOTE could also improve F1 scores where imbalance is most severe. Experimenting with more recent datasets and further hyperparameter tuning of the stronger models could yield additional performance gains beyond what was achieved in this study.

## REFERENCES


[1] R. Saborido Infantes, F. Khomh, G. Antoniol, and Y.-G. Guéhéneuc, "Comprehension of ads-supported and paid Android applications: Are they different?," in *Proc. IEEE Int. Conf. Program Comprehension (ICPC)*, 2017. doi: 10.1109/ICPC.2017.25.

[2] P. Pattanaik and D. Nagpal, "Comparison of machine learning algorithms used to catalog Google Appstore," *Journal of Medical Artificial Intelligence*, vol. 6, 2023, doi: 10.21037/jmai-23-58.

[3] R. Potharaju, M. Rahman, and B. Carbunar, "A longitudinal study of Google Play," *IEEE Transactions on Computational Social Systems*, pp. 1–15, 2017, doi: 10.1109/TCSS.2017.2732167.

[4] V. H. Tran, K. L. Bui, and D. L. Nguyen, "Analyzing user sentiment to rank English learning apps on Google Play," *HaUI Journal of Science and Technology*, vol. 61, no. 9E, pp. 9–15, 2025, doi: 10.57001/huih5804.2025.345.

[5] N. Khan, J. Khan, D. Khan, J. Li, M. Khan, and S. F. Khan, "A dataset of low-rated applications from the Amazon Appstore for user feedback analysis," 2026, arXiv preprint. doi: 10.48550/arXiv.2601.03009.

[6] A. K. Tegegnie, "Multi-task mining of Ethiopian mobile app reviews using machine learning and deep learning approaches," *International Journal of Interactive Mobile Technologies (iJIM)*, vol. 20, no. 1, pp. 137–159, 2026, doi: 10.3991/ijim.v20i01.58867.

[7] S. Sudianto, A. Sa'adah, and B. F. Arkana, "Utilization of adaptive machine learning for streaming sentiment analysis: The effects of batch and drift types," *Engineering, Technology & Applied Science Research*, vol. 16, no. 1, pp. 32384–32390, 2026, doi: 10.48084/etasr.16379.

[8] R. Maringka, A. Khoirunnita, E. Utami, and K. Kusnawi, "Android app rating classification on Google Play Store using random forest algorithm with SQL Server preprocessing," *TEPIAN*, vol. 2, pp. 79–84, 2021, doi: 10.51967/tepian.v2i2.404.

[9] S. Ramakrishnan, P. Kalaivani, B. Ashwin, S. Jaganeeshwar, and S. Senthilkumar, "Mobile application success prediction using machine learning," in *CEUR Workshop Proceedings*, vol. 3635, pp. 35–43, 2023.

[10] M.-K. Seo, O.-S. Yang, and Y.-H. Yang, "Global big data analysis exploring the determinants of application ratings: Evidence from the Google Play Store," *Journal of Korea Trade*, vol. 24, no. 7, pp. 1–28, 2020, doi: 10.35611/jkt.2020.24.7.1.

[11] Deepika, R. Kushwaha, and J. Boaddh, "An efficient machine learning technique for rating prediction of Google Play Store apps," *Journal of Computing Technologies (JCT)*, vol. 13, no. 8, pp. 1–9, 2024.

[12] M. Sai Kiran, P. V. S. Karthikeya, M. S. Kushal Raj, and M. Varun Teja, "Google Play Store app rating prediction," *International Journal of Creative Research Thoughts (IJCRT)*, vol. 10, no. 5, pp. c182–c191, 2022.

[13] A. Mueez, K. Ahmed, T. Islam, and W. Iqbal, "Exploratory data analysis and success prediction of Google Play Store apps," BRAC University, 2018. [Online]. Available: http://dspace.bracu.ac.bd

[14] M. W. Sarker, S. S. Al Nahian, A. M. Khan, A. Oraib, and S. M. Islam, "Rating detection by reviews using ML and NLP towards mobile phone recommendation," BRAC University, 2023. [Online]. Available: http://dspace.bracu.ac.bd:8443

[15] B. Jikadara, "Google Play Store Applications," Kaggle, [Online]. Available: https://www.kaggle.com/datasets/bhavikjikadara/google-play-store-applications. [Accessed: Apr. 19, 2026].